

Registo do Laboratório de Cibersegurança

Contents

Registo do Laboratório de Cibersegurança	1
Topologia do laboratório	1
Enquadramento nas certificações e normas em estudo	2
Entrada #1 — Scan nmap inicial ao Servidor Vulnerável	2
Entrada #2 — Verificação de serviços no Servidor Vulnerável	4
Entrada #3 — Instalação do Docker no Servidor Vulnerável	5
Entrada #4 — Arranque do container DVWA	6
Entrada #5 — Confirmação do container DVWA ativo	6
Entrada #6 — Inicialização e primeiro acesso ao DVWA	7
Entrada #7 — Login confirmado no DVWA	9
Entrada #8 — Scan nmap pós-DVWA (comparação com a Entrada #1)	10
Entrada #9 — Escolha do primeiro módulo de exploração	12
Entrada #10 — Teste normal do módulo SQL Injection (baseline)	12
Entrada #11 — SQL Injection bem-sucedida (nível Low)	13
Resumo da sessão — Dia 1 (2026-08-02)	15
Entrada #12 — <i>(a preencher)</i>	16
Screenshots	16

Registo do Laboratório de Cibersegurança

Este documento serve para registar, ao longo dos meses, os exercícios práticos realizados no laboratório de cibersegurança montado em VMware Workstation.

Cada entrada documenta: **objetivo/propósito, tipo de ataque ou técnica, comandos utilizados, resultado, como nos podemos defender**, e — quando aplicável — **o que correu mal ou falhou** no final da etapa.

Topologia do laboratório

- **Router/Firewall:** OPNsense (gateway 192.168.10.254)
- **Rede interna do lab:** LAN Segment “rede cyber” (192.168.10.0/24), isolada da rede de casa
- **Saída para a internet:** NAT configurado no OPNsense (WAN)
- **Máquinas do lab:**
 - Servidor Vulnerável (192.168.10.101)
 - Kali Atacante (192.168.10.102)
 - Windows Server
 - Windows 11
 - Ubuntu Desktop

Enquadramento nas certificações e normas em estudo

Cada entrada do registo passa a indicar, quando fizer sentido, a que domínio(s) das certificações/normas em estudo o exercício toca — para perceber que “setor” de conhecimento está a ser exercitado. Referência rápida dos domínios disponíveis:

Security+ (SY0-701): D1 Conceitos Gerais de Segurança · D2 Ameaças, Vulnerabilidades e Mitigações · D3 Arquitetura de Segurança · D4 Operações de Segurança · D5 Gestão de Programa de Segurança

CEH: D1 Fundamentos, Metodologia e Engenharia Social · D2 Reconhecimento, Scanning e Enumeração · D3 System Hacking e Malware · D4 Sniffing, Rede e Perímetro · D5 Web Server e Web Application Hacking · D6 Redes Sem Fios · D7 Mobile, IoT e OT · D8 Cloud Computing · D9 Criptografia

ISO/IEC 27001: D1 Estrutura da Norma · D2 Liderança e Planeamento · D3 Gestão de Risco e Declaração de Aplicabilidade · D4 Suporte e Operação · D5 Desempenho e Melhoria · D6 Controlos do Anexo A · D7 Auditoria e Certificação

NIS2: D1 Âmbito e Qualificação de Entidades · D2 Governação e Responsabilidade da Gestão · D3 Medidas de Gestão de Risco (Art. 21.º) · D4 Notificação de Incidentes (Art. 23.º) · D5 Supervisão, Execução e Sanções · D6 Cooperação Europeia · D7 Regime Jurídico Português

CompTIA A+ Core 1: D1 Dispositivos Móveis · D2 Redes · D3 Hardware · D4 Virtualização e Computação em Nuvem · D5 Diagnóstico de Hardware e Redes

CompTIA A+ Core 2: D1 Sistemas Operativos · D2 Segurança · D3 Resolução de Problemas de Software · D4 Procedimentos Operacionais

Nem todos os materiais são de cibersegurança pura (os A+ são mais de suporte técnico geral) — a integração só é feita quando o exercício toca genuinamente no domínio; não se força correspondência onde não existe.

Entrada #1 — Scan nmap inicial ao Servidor Vulnerável

Data/hora: 2026-08-02, 16:27 **Máquinas ligadas:** OPNsense, Kali Atacante (192.168.10.102), Servidor Vulnerável (192.168.10.101)

Objetivo / Propósito

Fazer o levantamento inicial (reconhecimento) do Servidor Vulnerável: descobrir que portas estão abertas, que serviços correm e que versões, para se ter uma baseline do estado do alvo antes de qualquer exercício de exploração.

Tipo de ataque / técnica

Reconhecimento (Reconnaissance) — a primeira fase de qualquer teste de intrusão (cf. Cyber Kill Chain / MITRE ATT&CK: Reconnaissance). Aqui especificamente: **port scanning** e **service/version detection** com nmap.

Comando executado

```
sudo nmap -sV -sC -p- 192.168.10.101
```

Resultado

```
Nmap scan report for 192.168.10.101
Host is up (0.00059s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.18 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 1e:a2:4c:c6:6d:d1:26:49:c9:b7:9a:fb:c3:dd:2c:6a (ECDSA)
|_  256 80:75:fc:43:b4:35:de:f9:60:ce:bc:a8:08:5c:2c:0d (ED25519)
MAC Address: 00:0C:29:B4:8B:9C (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Nmap done: 1 IP address (1 host up) scanned in 2.67 seconds

Observações e interpretação

- Apenas **1 porta aberta em 65535**: SSH (22/tcp), com **OpenSSH 9.6p1** — versão recente, sem vulnerabilidades públicas graves conhecidas associadas.
- Todas as restantes portas estão fechadas com reset ativo (o host responde mas recusa a ligação).
- Sistema identificado como Linux, sem outros serviços a expor mais detalhe sobre a distribuição/kernel exato.
- Scan rápido (2.67s) e sem latência (0.00059s), como esperado entre VMs na mesma máquina física.

Como nos podemos defender

- Manter o SSH atualizado (já está numa versão recente, o que é uma boa prática de base).
- Desativar login por password e usar apenas chaves públicas, para reduzir a superfície de brute-force.
- Restringir o acesso SSH por firewall (ex. regra no OPNsense que só permita a origem do Kali/administração, não toda a LAN).
- Usar fail2ban para bloquear tentativas repetidas de login falhado.

Domínios relacionados

- **CEH — D2 (Reconhecimento, Scanning e Enumeração)**: correspondência direta — este é exatamente o tipo de exercício desse domínio.
- **Security+ — D2 (Ameaças, Vulnerabilidades e Mitigações)**: o port scanning é uma técnica de deteção que se enquadra neste domínio.
- **A+ Core 1 — D2 (Redes)**: leitura de portas, serviços e protocolos de rede.

O que correu mal / faltou

Nada falhou tecnicamente — mas o resultado ficou aquém do esperado: para um “Servidor Vulnerável” de treino, ter apenas SSH exposto é invulgar. Isto levou a investigar (Entrada #2) e a perceber que a VM ainda não tinha nenhum software vulnerável instalado.

Próximos passos

- ☒ Confirmar se há serviços vulneráveis instalados mas inativos no Servidor Vulnerável → ver Entrada #2
- ☐ Repetir o scan após ativar/instalar esses serviços
- ☐ Explorar credenciais e superfícies de ataque no serviço SSH exposto (ex. brute-force controlado, se aplicável ao exercício)

Entrada #2 — Verificação de serviços no Servidor Vulnerável

Data/hora: 2026-08-02, 16:29 **Objetivo:** confirmar se existem serviços vulneráveis instalados mas inativos, explicando o resultado “pobre” do scan da Entrada #1.

Tipo de ataque / técnica

Não é um ataque — é uma etapa de **diagnóstico/preparação do ambiente** (auditoria local de serviços via systemctl), necessária antes de se poder montar qualquer exercício de exploração.

Comando executado

```
systemctl list-units --type=service --all
```

Resultado (resumo)

161 unidades carregadas, todas serviços base de um Ubuntu Server “de fábrica” (ssh, cron, Net-workManager, snapd, ufw, cloud-init, etc.). **Nenhum serviço de aplicação** encontrado — sem apache2, nginx, mysql/mariadb, vsftpd/proftpd, samba ou docker.

Observações e interpretação

- Confirma-se: o Servidor Vulnerável **não tem nenhum software vulnerável instalado** — não é um problema de serviços parados ou mal configurados, é uma VM Ubuntu limpa.
- docker.service nem sequer aparece na lista de unidades carregadas, o que sugere que o Docker também não está instalado.

Decisão: aplicação vulnerável a instalar

Depois de avaliar as opções (DVWA, Metasploitable, Juice Shop) do ponto de vista didático, ficou definido:

- **1ª fase:** DVWA (Damn Vulnerable Web Application), via Docker — níveis de dificuldade ajustáveis (Low/Medium/High/Impossible), mapeamento direto ao OWASP Top 10, alvo único e simples para consolidar a mecânica de “encontrar → confirmar → explorar”.
- **2ª fase (mais para a frente):** Metasploitable 2/3 — salto para exploração ao nível de rede/serviço (FTP, Samba, bases de dados) com o Metasploit Framework, quando a base estiver mais sólida.

Domínios relacionados

- **A+ Core 2 — D1 (Sistemas Operativos):** uso de systemctl para gerir e auditar serviços num sistema Linux.
- **Security+ — D4 (Operações de Segurança):** verificação de configuração/hardening antes de expor um alvo.
- **CEH — D1 (Fundamentos e Metodologia):** fase de reconhecimento interno/footprinting local do próprio alvo.

O que correu mal / faltou

O comando `sudo docker ps -a 2>/dev/null || echo "Docker não instalado"` confirmou “Docker não instalado” — como esperado, dado que docker.service não aparecia na lista de unidades. Ainda não foi possível confirmar `ss -tulnp` (por confirmar antes de instalar o Docker).

Próximos passos

- ☐ Confirmar `ss -tulnp` no Servidor Vulnerável (validação final de que nada está a correr)
 - ☐ Instalar Docker no Servidor Vulnerável
 - ☐ Instalar e arrancar o container DVWA
 - ☐ Repetir o scan `nmap` para confirmar a nova porta/serviço exposto
 - ☐ Começar exercícios de exploração pelo nível Low do DVWA
-

Entrada #3 — Instalação do Docker no Servidor Vulnerável

Data/hora: 2026-08-02 **Objetivo:** preparar o ambiente para correr o container DVWA (o Docker era a peça em falta, confirmada na Entrada #2).

Tipo de ataque / técnica

Não é um ataque — é preparação do ambiente (infraestrutura para o próximo exercício).

Comando(s) executado(s)

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo docker --version
```

Resultado

Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2

Observações e interpretação

Docker instalado e ativo (serviço já configurado para arrancar automaticamente com o sistema, via `enable --now`). Ambiente pronto para receber o container DVWA.

Domínios relacionados

- **A+ Core 2 — D1 (Sistemas Operativos):** gestão de pacotes (`apt`) e de serviços (`systemctl`) em Linux.
- **A+ Core 1 — D4 (Virtualização e Computação em Nuvem):** Docker como tecnologia de contentores.
- **ISO/IEC 27001 — D6 (Controlos do Anexo A):** relaciona-se com o controlo A.8.9 (gestão de configurações).

O que correu mal / faltou

Nada — instalação limpa, sem erros.

Próximos passos

- ☐ Arrancar o container DVWA (`docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa`)
 - ☐ Confirmar o container ativo (`docker ps`)
 - ☐ Inicializar a base de dados do DVWA via browser
 - ☐ Repetir o scan `nmap` para confirmar a nova porta 80 exposta
-

Entrada #4 — Arranque do container DVWA

Data/hora: 2026-08-02 **Objetivo:** disponibilizar o DVWA (Damn Vulnerable Web Application) no Servidor Vulnerável, para servir de alvo de exercícios de exploração web (OWASP Top 10).

Tipo de ataque / técnica

Preparação do ambiente (deployment da aplicação vulnerável) — ainda não é um exercício de ataque em si.

Comando(s) executado(s)

```
sudo docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa
```

Resultado

Imagem vulnerables/web-dvwa:latest descarregada com sucesso (8 camadas), container arrancado em modo detached, mapeado na porta 80 do Servidor Vulnerável.

```
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
91331b5820a183337c2957f3081ea89b1395ab9e549b7b6fa79e2dc5a07d28b0
```

Observações e interpretação

Container ativo com o ID acima. Falta confirmar com `docker ps` que está mesmo Up, e inicializar a base de dados via browser antes do primeiro uso.

Domínios relacionados

- **CEH — D5 (Web Server e Web Application Hacking):** preparação do alvo de aplicação web que sustenta este domínio.
- **A+ Core 1 — D4 (Virtualização e Computação em Nuvem):** deployment de container Docker.
- **ISO/IEC 27001 — D6 (Controlos do Anexo A):** relaciona-se com A.8.31 (separação de ambientes) — o DVWA corre isolado do resto do sistema, propositadamente vulnerável.

O que correu mal / faltou

Nada — download e arranque sem erros.

Próximos passos

- ☐ Confirmar `sudo docker ps` (estado Up)
- ☐ Aceder a `http://192.168.10.101/setup.php` a partir do Kali e clicar em “Create / Reset Database”
- ☐ Login inicial (admin / password)
- ☐ Repetir o scan nmap para confirmar a porta 80 exposta
- ☐ Iniciar exercícios de exploração pelo nível Low do DVWA

Entrada #5 — Confirmação do container DVWA ativo

Data/hora: 2026-08-02 **Objetivo:** verificar que o container DVWA arrancado na Entrada #4 está mesmo a correr e com a porta corretamente mapeada, antes de tentar aceder via browser.

Tipo de ataque / técnica

Verificação de ambiente (não é um ataque).

Comando executado

```
sudo docker ps
```

Para que serve: lista todos os containers Docker atualmente em execução (por defeito só mostra os ativos, ao contrário de `docker ps -a` que mostra também os parados). Serve para confirmar que o container arrancou de facto e não morreu logo a seguir (o que acontece com frequência se faltar alguma dependência ou configuração dentro da imagem).

O que se esperava encontrar: o container `dvwa` listado com estado Up e a porta 80 mapeada do container para o host (`0.0.0.0:80->80/tcp`).

Resultado

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
91331b5820a1	vulnerables/web-dvwa	"/main.sh"	49 seconds ago	Up	48 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp dvwa

Observações e interpretação

Resultado exatamente como esperado — sem surpresas. Container `dvwa` ativo há 48 segundos, porta 80 mapeada tanto em IPv4 (`0.0.0.0`) como IPv6 (`[::]`), o que significa que a aplicação já deve estar acessível via browser em `http://192.168.10.101/`.

Domínios relacionados

- **A+ Core 2 — D1 (Sistemas Operativos) / D3 (Resolução de Problemas de Software):** verificação de estado de um serviço em execução, base do diagnóstico técnico.

O que correu mal / faltou

Nada.

Próximos passos

- ☐ Aceder a `http://192.168.10.101/setup.php` a partir do Kali e clicar em “Create / Reset Database”
- ☐ Login inicial (admin / password)
- ☐ Repetir o scan nmap para confirmar a porta 80 exposta
- ☐ Iniciar exercícios de exploração pelo nível Low do DVWA

Entrada #6 — Inicialização e primeiro acesso ao DVWA

Data/hora: 2026-08-02 **Objetivo:** inicializar a base de dados do DVWA e confirmar que a aplicação está acessível e pronta a usar.

Tipo de ataque / técnica

Preparação do ambiente (não é um ataque) — último passo antes de o DVWA estar pronto para os exercícios de exploração.

Ação executada

Acesso via browser (Kali) a `http://192.168.10.101/setup.php`, seguido de “Create / Reset Database”.

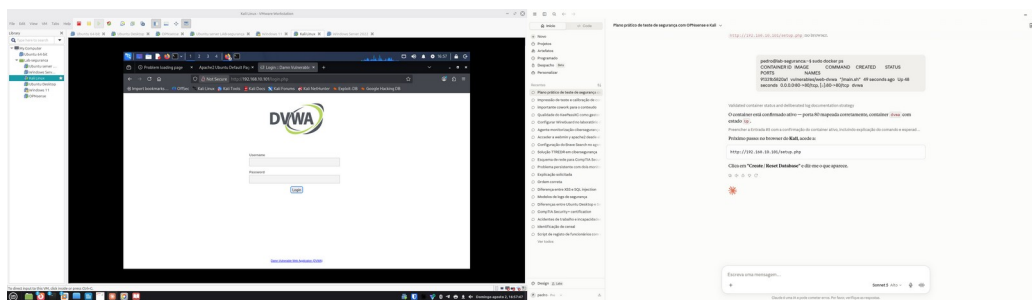
Para que serve: o `setup.php` do DVWA cria a base de dados MySQL/MariaDB interna ao container e as tabelas necessárias (utilizadores, registos de segurança, etc.) — sem isto, o login falha porque não há onde validar credenciais.

O que se esperava encontrar: após clicar em “Create / Reset Database”, o DVWA normalmente redireciona automaticamente para a página de login (`login.php`).

Resultado

O browser acabou na página `http://192.168.10.101/login.php`, com o logótipo do DVWA e os campos Username/Password visíveis — indicando que o setup decorreu como esperado.

Screenshot guardado:



Observações e interpretação

Sem surpresas — resultado igual ao esperado. Nota-se no separador do browser que houve uma tentativa anterior com “Problem loading page” (provavelmente antes do container estar totalmente pronto), resolvida ao tentar de novo.

Ainda por confirmar: login efetivo com as credenciais padrão (`admin / password`) — a validar na próxima entrada.

Como nos podemos defender

- As credenciais padrão do DVWA (`admin/password`) são propositadamente fracas para fins didáticos — em qualquer sistema real, credenciais padrão devem ser sempre alteradas no primeiro acesso.
- A página `setup.php` não deveria, num sistema em produção, ficar acessível publicamente — é uma superfície de ataque em si (permite recriar/apagar a base de dados).

Domínios relacionados

- **CEH — D5 (Web Server e Web Application Hacking):** preparação direta do alvo de exploração web.
- **Security+ — D2 (Ameaças, Vulnerabilidades e Mitigações):** credenciais padrão como vetor de ataque comum, mapeado no OWASP Top 10.

O que correu mal / faltou

Uma tentativa inicial deu “Problem loading page” (a confirmar a causa — possivelmente o container ainda não estava totalmente operacional nesse instante).

Próximos passos

- ☐ Fazer login com admin / password e confirmar acesso ao painel principal do DVWA
- ☐ Repetir o scan nmap para confirmar a porta 80 exposta
- ☐ Iniciar exercícios de exploração pelo nível Low do DVWA (começar por SQL Injection ou Command Injection)

Entrada #7 — Login confirmado no DVWA

Data/hora: 2026-08-02 **Objetivo:** confirmar que as credenciais padrão do DVWA dão acesso ao painel principal, validando que a aplicação está pronta para os exercícios de exploração.

Tipo de ataque / técnica

Verificação de ambiente (autenticação legítima com credenciais conhecidas, não é ainda um exercício de ataque).

Ação executada

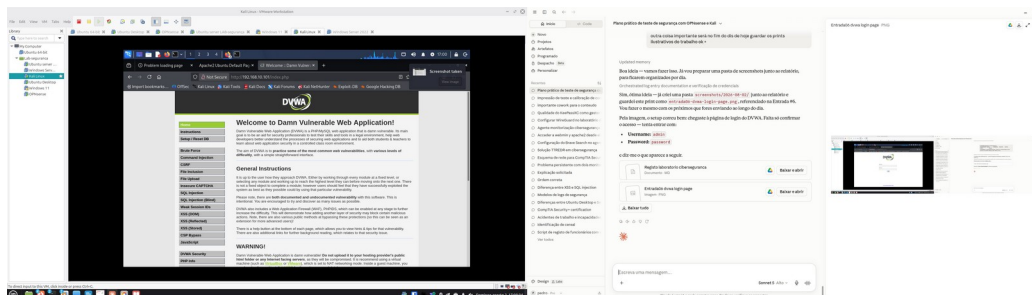
Login em `http://192.168.10.101/login.php` com admin / password.

O que se esperava encontrar: redirecionamento para a página inicial do DVWA (`index.php`), com o menu lateral de módulos de vulnerabilidade (SQL Injection, XSS, Command Injection, etc.).

Resultado

Login bem-sucedido — página “Welcome to Damn Vulnerable Web Application!” com o menu completo de módulos visível: Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM/Reflected/Stored), CSP Bypass, JavaScript, entre outros.

Screenshot guardado:



Observações e interpretação

Sem surpresas — resultado exatamente como esperado. O DVWA está agora totalmente operacional e pronto para os exercícios. A página confirma o objetivo do DVWA: praticar vulnerabilidades comuns em três níveis de dificuldade crescente.

Como nos podemos defender

- Este painel confirma visualmente a lista de vulnerabilidades típicas do OWASP Top 10 que vamos explorar uma a uma — cada módulo terá a sua própria secção de “como defender” quando o explorarmos.

Domínios relacionados

- **CEH — D5 (Web Server e Web Application Hacking):** o menu de módulos (SQLi, XSS, Command Injection, CSRF, File Upload/Inclusion) mapeia diretamente para este domínio.
- **Security+ — D2 (Ameaças, Vulnerabilidades e Mitigações):** cada módulo do DVWA corresponde a uma categoria do OWASP Top 10, referência central deste domínio.

O que correu mal / faltou

Nada.

Próximos passos

- ☐ Repetir o scan nmap (`sudo nmap -sV -sC -p- 192.168.10.101`) para confirmar a porta 80 agora exposta e comparar com a Entrada #1
 - ☐ Escolher o primeiro módulo de exploração (nível Low) para começar — sugestão: SQL Injection, por ser o mais representativo do OWASP Top 10
-

Entrada #8 — Scan nmap pós-DVWA (comparação com a Entrada #1)

Data/hora: 2026-08-02, 17:26 **Objetivo:** confirmar que a porta 80 (DVWA) passou a estar visível externamente e comparar com a baseline da Entrada #1.

Tipo de ataque / técnica

Reconhecimento (Reconnaissance) — mesma técnica da Entrada #1 (port scanning e service/version detection com nmap), agora repetida para medir a diferença no alvo.

Comando executado

```
sudo nmap -sV -sC -p- 192.168.10.101
```

O que se esperava encontrar: a porta 22 (SSH) igual à Entrada #1, mais a porta 80 (HTTP) agora aberta, servindo o DVWA.

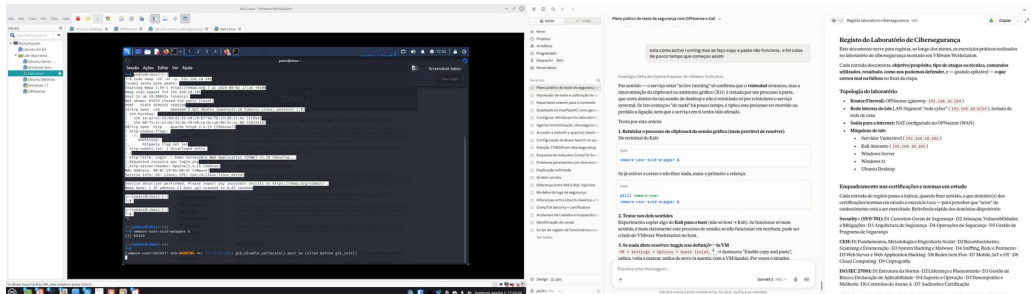
Resultado

```
Nmap scan report for 192.168.10.101
Host is up (0.00045s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.18 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 1e:a2:4c:c6:6d:d1:26:49:c9:b7:9a:fb:c3:dd:2c:6a (ECDSA)
|_  256 80:75:fc:43:b4:35:de:f9:60:ce:bc:a8:08:5c:2c:0d (ED25519)
80/tcp    open  http     Apache httpd 2.4.25 ((Debian))
| http-cookie-flags:
|   /:
|     PHPSESSID:
|       httponly flag not set
| http-robots.txt: 1 disallowed entry
|_/
|_http-title: Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Develop...
|_http-server-header: Apache/2.4.25 (Debian)
```

MAC Address: 00:0C:29:B4:8B:9C (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 8.83 seconds

Screenshot guardado:



Observações e interpretação

- **Confirmado:** a porta 80 apareceu, exatamente como esperado — o nmap já identifica o serviço como **DVWA v1.10** só pelo título da página (`http-title`), sem precisar de acesso prévio.
- **Surpresa útil:** o nmap já sinaliza sozinho uma vulnerabilidade real — a cookie `PHPSESSID` **não tem a flag `httponly`**. Isto significa que, se houvesse um XSS na aplicação, um script malicioso conseguiria ler essa cookie de sessão via JavaScript (`document.cookie`), o que normalmente `httponly` impede.
- **Apache httpd 2.4.25 (Debian)** é uma versão desatualizada (lançada por volta de 2016/2017) — típico de imagens Docker de treino, que fixam versões antigas de propósito para haver vulnerabilidades conhecidas a explorar.
- `robots.txt` tem uma entrada “disallowed”, o que por si só não é grave, mas mostra que o nmap também recolhe pistas de configuração do servidor web.

Como nos podemos defender

- Configurar a flag `HttpOnly` (e `Secure`) nas cookies de sessão, para que não sejam acessíveis via JavaScript — mitiga o roubo de sessão mesmo que exista um XSS.
- Manter o servidor web atualizado — uma versão de Apache de 2016/2017 tem múltiplas CVEs conhecidas e corrigidas em versões posteriores.
- Restringir o acesso a ficheiros como `robots.txt` só ao que é necessário divulgar, e nunca depender dele como controlo de acesso (é apenas uma convenção, não impede acesso real).

Domínios relacionados

- **CEH — D2 (Reconhecimento, Scanning e Enumeração):** deteção de serviço, versão e título de página via scripts NSE do nmap.
- **CEH — D5 (Web Server e Web Application Hacking):** a falha de cookie sem `httponly` é uma vulnerabilidade de aplicação web clássica.
- **Security+ — D2 (Ameaças, Vulnerabilidades e Mitigações):** gestão de sessão insegura é uma categoria do OWASP Top 10 (A05 — Security Misconfiguration / A07 — Identification and Authentication Failures).
- **Security+ — D3 (Arquitetura de Segurança):** configuração segura de cookies e hardening de servidores web.

O que correu mal / faltou

O copy/paste do Kali para o host deixou de funcionar a meio desta sessão (causa ainda não totalmente resolvida — open-vm-tools está active running, mas a sincronização do clipboard na sessão gráfica não). Contornado com screenshot; a resolver depois com mais calma.

Próximos passos

- ☐ Escolher e começar o primeiro módulo de exploração no DVWA, nível Low (sugestão: SQL Injection ou XSS, dado que já foi detetado o problema de httponly)
 - ☐ Investigar CVEs conhecidas para Apache 2.4.25 (Debian), como exercício complementar de reconhecimento
 - ☐ Resolver com mais calma o problema de clipboard do Kali (fora do âmbito do exercício de segurança em si)
-

Entrada #9 — Escolha do primeiro módulo de exploração

Data/hora: 2026-08-02 **Objetivo:** decidir por onde começar os exercícios de exploração no DVWA.

Decisão

SQL Injection, nível Low, pelas seguintes razões:

- É o módulo mais representativo do OWASP Top 10 e um dos vetores de ataque mais estudados em qualquer certificação (Security+, CEH), por isso rentabiliza melhor o tempo de estudo.
- O nível **Low** não tem qualquer proteção implementada — serve para primeiro *ver* a vulnerabilidade “em bruto” antes de progredir para Medium/High, onde o DVWA introduz filtros e defesas parciais que se aprende a contornar.
- É um bom “primeiro exercício” porque o resultado é visualmente óbvio (a aplicação devolve dados que não deveria), o que ajuda a consolidar a mecânica de “encontrar → confirmar → explorar” antes de módulos mais subtis como XSS ou CSRF.

Domínios relacionados

- **CEH — D5 (Web Server e Web Application Hacking):** SQL Injection é um dos ataques centrais deste domínio.
- **Security+ — D2 (Ameaças, Vulnerabilidades e Mitigações):** SQL Injection está no OWASP Top 10 (Injection).

Próximos passos

- ☐ No Kali, abrir o menu **SQL Injection** no DVWA (garantir que o nível de dificuldade está em **Low**, em DVWA Security)
 - ☐ Observar o formulário de pesquisa por User ID e testar o comportamento normal antes de tentar injetar
-

Entrada #10 — Teste normal do módulo SQL Injection (baseline)

Data/hora: 2026-08-02 **Objetivo:** confirmar o comportamento normal do formulário SQL Injection antes de tentar qualquer injeção.

Tipo de ataque / técnica

Nenhum ainda — uso legítimo da aplicação, para estabelecer a baseline de comparação.

Ação executada

No campo **User ID**, inserido o valor 1 e clicado em **Submit**.

O que se esperava encontrar: os dados de um único utilizador (o de ID 1).

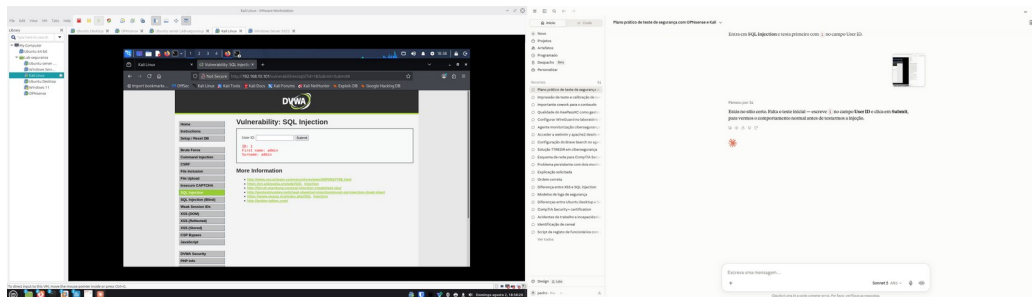
Resultado

ID: 1

First name: admin

Surname: admin

Screenshot guardado:



Observações e interpretação

Resultado exatamente como esperado — sem surpresas. A aplicação devolve um único registo por ID pedido, o que confirma que a query por trás do formulário filtra por `user_id`. Esta é a baseline que vai permitir perceber claramente se a próxima tentativa (injeção) altera o comportamento — por exemplo, se passar a devolver *vários* utilizadores em vez de um.

Domínios relacionados

- **CEH — D5 (Web Server e Web Application Hacking):** estabelecer comportamento normal antes de testar payloads é parte da metodologia de testes de aplicações web.

O que correu mal / faltou

Nada — ficou também resolvido o problema de rede do Kali que estava a bloquear o acesso ao DVWA (ver Entrada #9 → rede corrigida antes desta entrada).

Próximos passos

- ☐ Testar o payload de injeção `%' OR '1'='1` no campo User ID
- ☐ Comparar o resultado com esta baseline (1 utilizador vs. vários/todos)

Entrada #11 — SQL Injection bem-sucedida (nível Low)

Data/hora: 2026-08-02 **Objetivo:** confirmar a vulnerabilidade de SQL Injection no formulário User ID, comparando com a baseline da Entrada #10.

Tipo de ataque / técnica

SQL Injection (in-band / UNION-independent) — injeção de uma condição sempre verdadeira (tautologia) para contornar o filtro da query. OWASP Top 10: A03 Injection.

Ação executada

No campo **User ID**, inserido:

```
%' OR '1'='1
```

Para que serve: a query original do DVWA (nível Low) é algo como `SELECT first_name, last_name FROM users WHERE user_id = '$id';`, sem qualquer sanitização do valor introduzido. Ao fechar a aspa (%) e acrescentar `OR '1'='1`, a condição `WHERE` passa a ser sempre verdadeira para **todas** as linhas, não só para o ID pedido.

O que se esperava encontrar: a lista completa de utilizadores da tabela, em vez de um único registo.

Resultado

```
ID: '%' OR '1'='1  
First name: admin  
Surname: admin
```

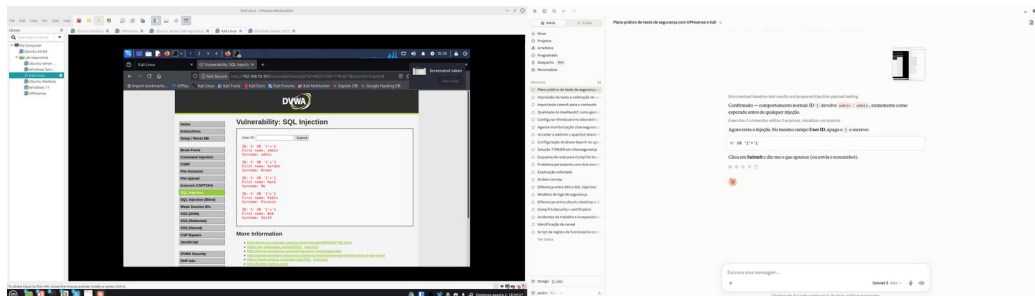
```
ID: '%' OR '1'='1  
First name: Gordon  
Surname: Brown
```

```
ID: '%' OR '1'='1  
First name: Hack  
Surname: Me
```

```
ID: '%' OR '1'='1  
First name: Pablo  
Surname: Picasso
```

```
ID: '%' OR '1'='1  
First name: Bob  
Surname: Smith
```

Screenshot guardado:



Observações e interpretação

- **Confirmado, sem surpresas:** a aplicação devolveu **5 utilizadores** em vez de 1 — prova clara e visual de que a injeção funcionou e de que a query não está a sanitizar/parametrizar o input.

- Este resultado revela também os **nomes de utilizador da base de dados** (admin, gordonb, 1337, pablo, bob são os logins típicos do DVWA associados a estes nomes) — informação valiosa para um atacante tentar depois um ataque de força bruta ou de credenciais.
- A query real por trás confirma-se como vulnerável por **concatenação direta de string**, em vez de usar *prepared statements / parameterized queries*.

Como nos podemos defender

- **Prepared statements / parameterized queries:** a defesa fundamental — o valor introduzido pelo utilizador nunca é interpretado como parte do código SQL, apenas como dado.
- **Validação de input:** neste caso o campo espera um ID numérico; rejeitar qualquer valor que não seja um número inteiro já bloquearia este payload.
- **Princípio do menor privilégio na base de dados:** a conta usada pela aplicação para consultar esta tabela não devia ter permissões além do estritamente necessário (ex. sem DROP, DELETE se não for preciso).
- **WAF (Web Application Firewall):** o próprio DVWA menciona ter um PHPIDS que pode ser ativado para bloquear padrões como este — uma camada adicional, não substituta da correção na aplicação.

Domínios relacionados

- **CEH — D5 (Web Server e Web Application Hacking):** exercício central deste domínio — SQL Injection clássica.
- **Security+ — D2 (Ameaças, Vulnerabilidades e Mitigações):** Injection é a categoria A03 do OWASP Top 10, referência central deste domínio.
- **ISO/IEC 27001 — D6 (Controlos do Anexo A):** relaciona-se com A.8.28 (codificação segura) — prepared statements são uma prática direta desse controlo.

O que correu mal / faltou

Nada — a injeção correu exatamente como previsto na Entrada #9.

Próximos passos

- ☐ Subir o nível de dificuldade do DVWA para **Medium** e repetir o mesmo payload, para ver que proteção (parcial) foi introduzida e como contorná-la
- ☐ Documentar a diferença de comportamento entre Low e Medium como próxima entrada
- ☐ Mais tarde, explorar SQL Injection (Blind) como exercício complementar

Resumo da sessão — Dia 1 (2026-08-02)

O que foi feito: - Definida a topologia do laboratório (OPNsense + Kali + Servidor Vulnerável) e resolvidos os problemas iniciais de rede e acesso SSH. - Scan nmap inicial ao Servidor Vulnerável — só SSH exposto (Entrada #1). - Investigação confirmou que a VM não tinha nenhum software vulnerável instalado (Entrada #2); decisão de usar **DVWA** como alvo didático, com Metasploitable como plano para mais tarde. - Instalado o Docker e arrancado o container DVWA (Entradas #3-#5); base de dados inicializada e login confirmado (Entradas #6-#7). - Scan nmap pós-DVWA revelou a porta 80 aberta e uma vulnerabilidade real detetada automaticamente pelo nmap: cookie de sessão sem a flag httponly (Entrada #8). - Resolvidos, em paralelo, dois problemas recorrentes: a placa de rede do Kali a reverter para a rede de casa, e uma falha temporária do clipboard entre o host e o Kali (este último tratado numa conversa à parte). - **Primeiro exercício de exploração:** SQL Injection no nível Low do DVWA, bem-sucedida — a aplicação passou de devolver 1 utilizador a devolver os 5 da tabela, com o payload `% OR '1'='1` (Entradas #9-#11).

Estado do laboratório no fim do dia: DVWA a correr e acessível em <http://192.168.10.101/>, com o módulo SQL Injection (nível Low) já explorado com sucesso.

Para retomar: subir o DVWA para o nível **Medium** e repetir o mesmo payload de SQL Injection, para comparar que proteção foi introduzida e como a contornar.

Entrada #12 — (a preencher)

Data/hora: **Objetivo:** **Tipo de ataque / técnica:** **Comando(s) executado(s):** **Resultado:**
Como nos podemos defender: **Domínios relacionados:** **Observações e interpretação:** **O que correu mal / faltou:** **Próximos passos:**

Screenshots

Os prints ilustrativos de cada dia de trabalho ficam guardados em `screenshots/AAAA-MM-DD/`, referenciados a partir da entrada correspondente.

- `screenshots/2026-08-02/entrada06-dvwa-login-page.png` — página de login do DVWA após inicialização da base de dados
- `screenshots/2026-08-02/entrada07-dvwa-welcome-page.png` — página inicial do DVWA após login bem-sucedido
- `screenshots/2026-08-02/entrada08-nmap-pos-dvwa.png` — resultado do scan nmap pós-DVWA, mostrando a porta 80 aberta
- `screenshots/2026-08-02/entrada10-sqli-teste-normal.png` — teste normal do módulo SQL Injection (ID 1 → admin/admin)
- `screenshots/2026-08-02/entrada11-sqli-injecao-bem-sucedida.png` — injeção SQL bem-sucedida, devolvendo todos os 5 utilizadores